



Enhancing network security through integration of game theory in software-defined networking framework

Florea Razvan¹ · Craus Mitica²

Published online: 19 March 2025
© The Author(s) 2025

Abstract

This paper presents a comprehensive study on the implementation of a Security-Enhanced Software-Defined Networking (SEC-SDN) framework integrated with game theory modeling and Snort, an open-source intrusion detection system, to strengthen cloud environments against Distributed Denial-of-Service (DDoS) attacks. As cloud computing continues to dominate the technological landscape, ensuring robust security for these highly scalable yet vulnerable systems is critical. The proposed SEC-SDN framework incorporates game theory to model dynamic interactions between attackers and defenders, enabling proactive and adaptive decision making for threat mitigation. By integrating Snort with SEC-SDN, the framework gains enhanced monitoring capabilities, facilitating real-time detection of malicious traffic patterns and the immediate enforcement of security policies to neutralize threats. This synergistic approach not only limits the impact of DDoS attacks but also preserves the availability and integrity of cloud services. A series of simulated DDoS attack scenarios were conducted on a cloud-based network to evaluate the effectiveness of the proposed framework. The results demonstrate significant improvements in threat detection, response times, and overall resilience of cloud environments against cyber threats. The findings highlight the potential of combining SEC-SDN, game theory modeling, and intrusion detection systems such as Snort to establish a scalable, adaptive, and robust security framework. This study contributes to the advancement of cloud security by proposing an innovative solution that dynamically evolves to address the increasingly sophisticated tactics of cyber attackers, ensuring the reliability and continuity of cloud services.

Keywords Software-defined network · SEC-SDN · Cloud security · Real-time threat mitigation · Game theory

1 Introduction

Modern networks' increasing complexity and scale demands innovative solutions for effective threat detection and response. Traditional network architectures face limitations in adaptability and responsiveness to emerging security threats. With its centralized control and programmable infrastructure, Software-defined network (SDN) offers a promising approach to overcome these challenges. This research inves-

tigates the integration of SDN and Game Theoretic Strategies to create a more responsive and efficient network security framework.

The emergence of virtualization technology is a historic breakthrough in network development and a major change in the age of the information [1]. Network researchers have difficulty making breakthroughs in the deployment of existing network architectures and the further development of the network encountered obstacles. The need for a new network architecture has become imminent, and the control and forwarding of separate networks have been mandatory [2].

Software-defined network is an emerging network architecture widely adopted in both the IT industry and academic circles for a variety of purposes, including monitoring, security, cloud computing, big data, virtualization, and WAN optimization. The SDN architecture is divided into three distinct layers: the infrastructure layer, the control layer, and the application layer. One of the key innovations of SDN is the decoupling of the control plane from the data plane,

✉ Florea Razvan
razvan.florea@student.tuiasi.ro

Craus Mitica
mitica.craus@academic.tuiasi.ro

¹ Faculty of Automatic Control and Computer Engineering, "Gheorghe Asachi" Technical University of Iasi, 700050 Iasi, Romania

² Department of Computer Science and Engineering, Faculty of Automatic Control and Computer Engineering, "Gheorghe Asachi" Technical University of Iasi, 700050 Iasi, Romania

which provides several benefits over traditional networking architectures, such as enhanced manageability, scalability, programmability, and performance improvements [3]. This separation allows the network controller to have a comprehensive overview, giving it the ability to monitor and manage all network activities effectively.

The increasing adoption of SDN in cloud environments has revolutionized network management by enabling dynamic programmability and centralized control. However, this flexibility also introduces new security challenges, particularly in mitigating DDoS attacks, which remain one of the most prevalent threats to network stability. To address these challenges, this research introduces the SEC-SDN framework, an intelligent and adaptive solution designed to improve the resilience of SDN-based networks against DDoS attacks.

The SEC-SDN framework leverages the dynamic capabilities of SDN technology to implement real-time threat detection and mitigation strategies. By integrating game-theoretic principles, the framework models advanced defense mechanisms such as honeypots and dynamically reconfigures network defenses based on evolving threats. This adaptive approach enables the framework to accurately identify the source of an attack while minimizing the depletion of critical network resources, such as bandwidth, during mitigation efforts.

Despite its potential, the SEC-SDN framework faces several challenges. Scalability remains a key concern, particularly in large-scale networks where dynamically managing security measures across numerous devices adds significant complexity. Additionally, the continuous monitoring and reconfiguration required by the framework may introduce resource overhead, particularly in environments with constrained computational or bandwidth resources. Furthermore, while the framework is adept at addressing known attack patterns, its adaptability to novel and unforeseen threats remains an area for improvement. Lastly, the practical implementation of the framework in real-world environments can be complex and requires extensive integration efforts with existing network infrastructures. This work aims not only to contribute to the academic understanding of SDN-based security, but also to provide a practical foundation for implementing resilient security measures in modern cloud infrastructures.

In Sect. 2, we present the results of similar works on SDN and proposed frameworks trying to make a comparison between them and what we propose. In Sect. 3, we briefly present the parts involved in our work like SDN Technology, OpenFlow, SDN Controller trying to highlight the importance and the broad usage of this technology. In Sect. 4, we present SEC-SDN and how one can improve performance against DDoS attacks in SDN environments. Our aim is to develop a model that can accurately identify the source of

an attack, determining whether it originates from a host or a switch, while ensuring that mitigation efforts do not unduly deplete network resources, such as bandwidth. In Sect. 5, we sought to explore the mathematical and conceptual integration of a game-theoretic approach into the cybersecurity framework using SDN. By proposing a mathematical model that correlates the strategic defense mechanisms involving honeypots, as derived from game theory, with the dynamic security configurations enabled by SDN, our aim is to establish a comprehensive security framework. This framework is designed to enhance the network's resilience by intelligently anticipating potential threats and dynamically reconfiguring network defenses in real-time. Through this integration, we strive not only to improve the effectiveness of cybersecurity measures but also to provide a scalable and flexible solution that can adapt to the ever-changing landscape of cyber threats. In Sect. 6, we implement and evaluate the SEC-SDN framework and highlight the results in real SDN environments using the Mininet emulator and different DDoS tools to generate normal traffic, UDP flooding, and also DDoS attack traffic. In Sect. 7, we present the conclusions of this work and what we can improve in future research.

2 Related work

Transitioning from an in-depth exploration of defending OpenStack cloud platforms against low-rate DDoS attacks through Software-Defined Networking to enhancing network security by integrating SDN with a proposed framework marks a pivotal shift towards a more dynamic and comprehensive approach to cybersecurity. Initial research [3] underscores the critical vulnerabilities that cloud environments face, particularly in the context of low-rate DDoS attacks that often go undetected due to their subtlety and the complex nature of cloud architectures. It highlights the innovative use of SDN to centralize control and enable a more nuanced and responsive defense mechanism against such insidious threats. The proposed framework [3] addresses the challenge of detecting and defending against Low-rate Distributed Denial of Service (LDDoS) attacks in cloud environments, particularly those utilizing SDN. Introduces a unified detection and defense strategy that leverages the centralized control capabilities of SDN to effectively manage network traffic. The framework comprises three main components: data acquisition preprocessing, attack detection, and defense mechanisms. By monitoring traffic flow through Open vSwitch, the system identifies potential attack traffic based on specific characteristics, such as the average and maximum lengths of data packets. Detected attacks are then mitigated through policies issued by a central controller, which block malicious traffic and ensure the continued availability of services. This approach aims to enhance the security

of cloud platforms like OpenStack by providing a flexible and modular solution to combat low-rate DDoS attacks, without impairing the functionality of nontargeted system modules.

In a previous study [17], we focused on a game-theoretic approach to network security using honeypots, introducing a strategic framework that analyzes the interactions between attackers and defenders through the lens of game theory, specifically employing the Stackelberg Equilibrium (SE) to model these interactions. It outlines a comprehensive mathematical formulation that captures the dynamics of cyber threats and the strategic deployment of honeypots as decoys to entrap attackers, thereby enhancing the security posture of the network.

The relationship between previous research is established through the shared goal of enhancing network security through innovative and flexible strategies. The game-theoretic approach [17] offers a theoretical and strategic framework that can underpin the dynamic, responsive actions facilitated by the SDN framework [3] and the current research. Essentially, strategic insights derived from game theory can inform decision-making processes within an SDN-enabled network, guiding dynamic configuration changes to effectively counteract and mitigate cyber threats.

Moreover, the integration of game-theoretic strategies with SDN's capabilities can lead to the development of more sophisticated, adaptive security mechanisms. For example, the strategic placement and use of honeypots, as discussed in [17], can be dynamically managed through SDN's centralized control mechanisms, allowing for real-time adjustments in the network's defense posture based on the evolving threat landscape. This integration not only enhances the effectiveness of each approach individually, but also creates a more resilient, intelligent network defense system that leverages the strengths of both strategic game theory and advanced networking technologies.

The new and innovative aspect of this approach lies in its ability to seamlessly adapt to evolving security threats. The SDN centralized control model offers unparalleled visibility and management of network traffic, enabling rapid deployment of security policies and rules across the network. Furthermore, this synergy allows for the dynamic reconfiguration of network resources in response to detected threats, enhancing the resilience and robustness of network infrastructures against sophisticated attacks. The adaptability of the combined SEC-SDN framework ensures that security measures evolve in lockstep with emerging threats, providing a proactive approach to network security rather than a reactive one.

In the landscape of SDN security frameworks aimed at thwarting DDoS attacks, several notable efforts precede the SEC-SDN framework, each with unique approaches to detect and mitigate these threats. These frameworks leverage SDN's centralized control and programmability to enhance network

security, but differ in methodologies, focus areas, and technologies used. Here is a comparison of SEC-SDN with some of these frameworks presented in Table 1.

Dao et al. [12] introduced a method within a network controller for monitoring packets by their IP addresses to identify DDoS attacks. The approach treats all incoming requests to the controller as potentially suspicious. For each request, the controller creates a temporary flow entry in the switch based on the request's source IP address. This entry is assigned a brief lifespan to ensure its quick removal. The controller also examines the frequency of hits on this entry, using a predefined threshold to discern between normal and malicious requests. If a flow entry is seldom used (indicating a hit rate lower than that of standard entries), the source IP is flagged as hostile. Consequently, packets from this source are blocked at the switch, thus minimizing the number of flow entries and safeguarding the controller-switch bandwidth. Mousavi et al. [13] adopt entropy as a means to detect DDoS attacks, leveraging its capacity to quantify randomness. This method involves monitoring the entropy over a set time frame and comparing it against a threshold. An attack is suspected if the entropy falls below this threshold for five consecutive intervals. However, this detection mechanism can be circumvented if an attacker alters both source and destination IP addresses, which could result in entropy levels that mimic regular network traffic. The calculation of entropy increases with the diversity of source and destination IP addresses involved. Dharma et al. [14] suggest the implementation of a "flow collector" positioned between the network switch and controller to sieve out malicious traffic. This flow collector activates to scrutinize suspicious packets more closely once the quantity of invalid packets surpasses a certain threshold within a specified timeframe. Shoeb et al. [15] introduce the concept of "peak time" and develop a trust level system to protect both the control and data planes from DDoS attacks. This system utilizes the trust level of a node to prioritize its processing on the controller, assigning values based on the node's behavior outside of peak times. During periods of high traffic, the controller rejects requests from nodes that have exceeded a predetermined request threshold. Additionally, even for nodes considered normal, the controller enforces a new rule on the switch that has a reduced timeout period. Macedo et al. [16] introduced a framework based on a multi-controller cluster model comprising three stages: detection, election, and composition. In the detection stage, the model assesses the controller for signs of overload by monitoring the delay in control messages, stability, and the impact on switches. During the election stage, criteria such as processing power, memory capacity, and network latency are used to define the performance level. This stage also prepares for a scenario where the leading controller fails during a DDoS attack by selecting a backup leader and multiple candidates based on their performance levels. Finally,

Table 1 Existing SDN framework detection and defence solutions

Algorithm	Detection or defence	Strength	Weaknesses
Dao et al. [5]	Both	Feasible in small networks	High resource consumption when attacks
Mousavi et al. [6]	Detection	Less resource consumption for a short time detection	The threshold of detection is hard to define
Dharma et al. [7]	Both	High detection rate	Legitimate users have delays because of false positives
Shoeb et al. [8]	Both	The flow table is protected	Hard to define peak time
Macedo et al. [9]	Both	Has multiple controllers associated	Continual controller election may occur in high attacks
Wang et al. [10]	Both	Minimal forwarding rules	Does not address the dynamic threshold
Gala et al. [11]	Both	The use of autoencoders combined with Snort	More generalized and does not offer granularity

the composition stage employs a genetic algorithm to identify the most efficient clustering arrangement, optimizing the network's resilience and response to DDoS threats. Wang et al. [4] focuses on protecting the SDN control plane against flood attacks using Floodguard framework. It introduces a proactive defense mechanism that preinstalls minimal forwarding rules to reduce the number of Packet-In messages during an attack. FloodGuard also utilizes a dynamic data plane anomaly detection system to identify and mitigate flooding threats. Unlike SEC-SDN, FloodGuard emphasizes pre-attack preparation and control-plane protection without explicitly addressing the dynamic threshold adjustment for Packet-In messages based on real-time traffic analysis. Shin et al. [5] designed SPIFFY to detect and mitigate DDoS attacks in real-time by monitoring the flow setup rate within the network. By setting thresholds on the number of flow setup requests, SPIFFY can quickly identify potential DDoS activities. However, SPIFFY's approach is more generalized in detecting rate anomalies and does not offer the granularity of SEC-SDN's method, which analyzes controller statistics for a more precise identification of attack sources.

SEC-SDN differentiates itself by focusing specifically on the detection and mitigation of flooding DDoS attacks through a nuanced analysis of controller statistics. It proposes a novel mechanism for dynamically adjusting the threshold of Packet-In messages, enabling a more precise and efficient detection and mitigation of DDoS attacks. This approach allows SEC-SDN to adapt to varying network conditions and attack patterns without overburdening network resources. Furthermore, SEC-SDN's strategy for identifying the attack origin-whether from a host or a switch-enables targeted mitigation measures, conserving network bandwidth and maintaining service quality during an attack. This level of specificity and resource efficiency in response to DDoS threats marks a significant advancement over previous frameworks.

3 Materials and methods

3.1 Research background

Network security has been a critical topic of discussion and research since the inception of computer networking, its primary goal being to ensure the protection of data and core information. Legal use, especially with the widespread use of the Internet, network security is very important for both individual users and businesses. Cybersecurity is only a relative concept, not an absolute one [6]. It can be said that there is no absolute network security because the network is limited by various factors, the network security cannot be absolute in any form. In a non-virtual environment, the operating system is operated and run through hardware. The

system operates on hardware, with the allocation and management of hardware resources handled by the operating system, which utilizes the resources of the entire machine. Unlike virtual environments, the operating system does not run directly on the hardware and is resource sensitive. Usage and management are managed and scheduled by the hypervisor, and in a virtual environment, a hypervisor can be at the same time running multiple virtual machines, each virtual machine being shared with each other for the resources used by the underlying hardware. With the continuous application of virtualization technology.

The security problems brought about by virtualization technology cannot be ignored, and security authentication and access control technology are important technologies for ensuring network security. It has been widely used in traditional networks and has achieved certain results. Traditional access control mechanisms are built on traditional networks. Based on the construction, most of them are based on access control to physical ports, and one physical port is opposed to multiple virtualization ports in a virtualization environment. In addition, traditional access control technology cannot achieve fine control of each port in virtualization technology, giving network security bands in the virtualization environment [6].

3.2 SDN technology

SDN Basic Architecture is a concept proposed when designing for the drawbacks of traditional networks, and is also a new network architecture [7]. In traditional networks, the transmission of the network is based on network equipment, network equipment is usually designed to rely on market needs, and to improve production efficiency, the manufacturer usually adopts and integrates a new design approach. Switches and directors are essential equipment for traditional networks, and they are designed with the traditional scale in mind. The integrated design has great limitations in device performance, and the user is limited to network equipment operation and cannot access the control of the network device line, which brings many disadvantages to user use and network development [8].

Unlike traditional networks, SDN networks enable the control and forwarding of separation. In this architecture, the switch is only responsible for forwarding data, and the control of data is mainly entrusted to the control plane; therefore, the overall efficiency of the network is not affected by the performance of network devices. In the network structure, communication between the upper layer application and the underlying network device can be realized in the form of software programming, and the network will become more flexible and convenient [9].

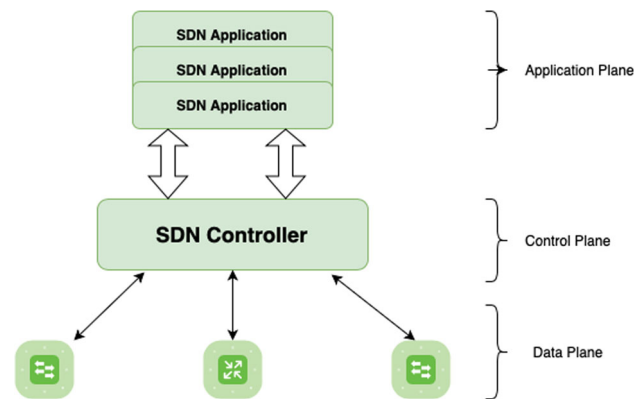


Fig. 1 SDN architecture

In Fig. 1 we emphasize the main part of an SDN Architecture which consists of the application plane, the control plane, and the data plane.

The application layer is located on the top layer, and there are different applications and services in the application layer. In terms of network operation and network security, many functions and services, such as load balancing, management, and monitoring of network performance, are represented by software applications.

The control layer is located in the middle layer, and the controller is the main component of the control layer, and the main function is to realize the data plane. The sources are properly orchestrated, and the network topology information is viewed and maintained. There can be multiple controllers in an SDN network, and one controller can connect and control multiple devices. The controller acts as a platform on the SDN network and can be switched on and off with the SDN. A line session that provides a programming interface for an application.

The lowest layer is the infrastructure layer, and the basic infrastructure layer is usually composed of SDN switches and OpenFlow protocols. The main function is to be responsible for data processing, forwarding, and status collection.

3.3 OpenFlow

The OpenFlow switch is one of the important components that make up the SDN network. An OpenFlow switch typically consists of three parts: a flow table, a secure channel, and the OpenFlow protocol. OpenFlow switches differ from traditional Switches in functionality, that is, OpenFlow switches are only responsible for forwarding data. In this article, OpenvSwitch software is used as OpenFlow switches, which are described here. OpenvSwitch (OVS) is currently a virtual machine with more applications in SDN networks, which complies with the Apache 2.0 license, which is the base for implementing its functionality in software. The design of OVS is based on OpenFlow, an open-source, multi-platform

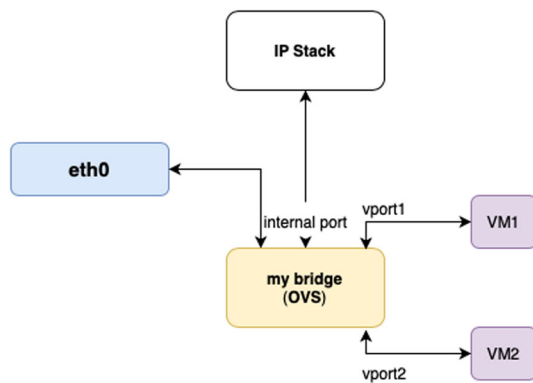


Fig. 2 OVS architecture

virtual intercourse swap, that supports OpenFlow protocol, closely connected to the controller. If the controller also supports the OpenFlow protocol, the user can operate the operation and use this type of controller to remotely control the OVS.

Although OVS is a virtual switch, it is different from traditional physical switches, but the principle of operation is similar. In the specific implementation, the virtual machine is connected to the physical network card and the corresponding virtual network card on the computer. When data transmission is carried out, it is mainly implemented through the virtual machine link, which corresponds to the MAC address carried by the data as seen in Fig. 2.

When a packet is sent, the packet first passes through the virtual NIC, which is configured according to the virtual machine, and the packet passes through the virtual After the NIC, the packet is forwarded to the virtual switch. The OVS switch is based on OpenFlow, which is provided for its support capability when a packet arrives at the OVS switch because the OVS switch itself saves the corresponding flow table, so the data will be treated first. The package performs a lookup match for the local flow table and if the corresponding flow table is found to match, the packet will be processed according to the instructions corresponding to the flow table to operate [7]. If there is no corresponding match, the OVS switch will send the packet to the controller, which will process and control it. The server will formulate the corresponding flow table based on the valid data of the packet and deliver the flow table to the switch. Similarly, the forwarding of packets can pass through physical NICs, because one end of the OVS switch is connected to a physical NIC, and if forwarding is required through a physical NIC, only packets need to be sent to the physical NIC and finally forwarded to an external network device connected to the physical NIC of the computer.

3.4 SDN controller

SDN represents a significant shift in network architecture, focusing on centralizing and abstracting control logic to simplify network management and increase flexibility. At the heart of this architecture is the SDN controller, which acts as the central brain of the network, orchestrating traffic flows and policies from a centralized point.

Centralized Control and Network Management: SDN controllers centralize network intelligence, abstracting the control plane from the data plane. This separation allows for more straightforward network adjustments and policy implementations across various hardware and vendor systems. Centralized control improves the flexibility and scalability of the network, allowing quick adaptation to changing requirements and conditions without the need for manual configuration on individual network devices [22].

The performance of an SDN controller is crucial as it affects the entire network's efficiency. Controllers must manage multiple requests and maintain a universal network view, supporting rapid decision-making and enforcement. Advanced controllers optimize these operations through improved algorithms and data structures, which help to handle high throughput and low latency demands efficiently [23].

Challenges in Controller Placement: Determining the optimal placement of SDN controllers is critical, as it affects network latency, fault tolerance, and overall performance. Various studies propose algorithms to address the controller placement problem, focusing on minimizing latency and balancing load among controllers. An effective placement of the controller ensures robust network performance and reliability, particularly in large-scale environments [24].

Ryu is an open source project led by the Japanese company NTT, which literally means "Flow" in Japanese. The Ryu scheduling target is to provide an SDN operating system with logically centralized control with a well-designed API interface for network applications and easily create new management and control applications. Ryu is written in Python, fully compliant with the Apache license, and able to support other versions of the OpenFlow protocol. The Ryu architecture Fig. 3 and the SDN architecture fit perfectly. The control layer mainly provides control capabilities, passes the REST API over the northbound interface, and serves SDN apps for scheduling and controlling traffic and the network. Through the southbound interface, protocols such as OpenFlow control the OpenFlow switch and complete traffic interaction. Among them, the Ryu control layer plays the role of connecting the upper and lower levels, which is the north-facing Control and switching hub for interfaces.

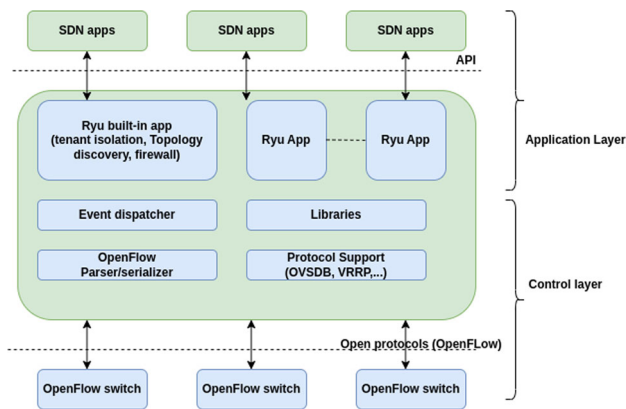


Fig. 3 Ryu controller

3.5 SNORT IDS

Intrusion Detection Systems (IDS), such as Snort, are essential for the security infrastructure of networked environments. Their implementation within a SDN framework, specifically SEC-SDN, leverages advanced techniques for more effective network monitoring and threat detection.

An approach to integrating Snort IDS in the SEC-SDN framework involves the use of an automatic rule generation technique, which simplifies the creation of detection rules through automation, minimizing human error and resource expenditure. This is complemented by a security event correlator that efficiently processes and reduces false alarms, thereby enhancing the overall effectiveness of the network security system. The combination of Snort rule-based detection with automated rule generation offers a robust defense mechanism against a variety of network threats [18].

Further enhancing Snort IDS's capabilities within SEC-SDN, collaborative techniques involving deep neural networks (DNN) have been employed. These techniques utilize DNNs to analyze network traffic after initial detection by Snort, thus significantly reducing false positives and improving the accuracy of threat detection. This dual layer detection strategy ensures a more reliable and scalable network security solution [19].

Moreover, the SEC-SDN framework can incorporate elastic management of virtualized Snort instances, optimizing the distribution of network traffic, and enhancing response times. Using control theory principles, such as Proportional Integral (PI) and Proportional Integral Derivative (PID) controllers, the system can dynamically adjust the load across Snort instances based on real-time network conditions, thus maintaining high detection performance even under fluctuating network loads [20].

Additionally, the use of blockchain technology within the SEC-SDN architecture supports a collaborative intrusion detection network by ensuring secure and transparent sharing

of IDS alerts and signatures across distributed Snort nodes. This improves trust and coordination among nodes, leading to faster and more accurate detection of network threats [21].

By adopting these innovative methods, Snort IDS's implementation within the SEC-SDN framework not only enhances the security posture of networks, but also brings scalability, reliability, and efficiency to intrusion detection and response efforts.

3.6 Importance of other programming language in SDN

The programming language P4 (Programming Protocol Independent Packet Processors) has become a key tool in the evolution of SDN due to its ability to provide enhanced programmability and flexibility at the data plane. Unlike traditional approaches such as OpenFlow, which rely on fixed function protocols, P4 allows developers to define custom packet processing behaviors directly at the data plane. This capability enables application-specific optimizations and improved network adaptability, making P4 integral to the advancement of SDN technologies. Recent studies have highlighted the role of P4 in enabling a protocol-agnostic, target-independent approach to network configuration and optimization [25].

A key advantage of P4 is its ability to support data-plane programmability, allowing developers to deploy application-specific functionalities without requiring changes to the control plane. This feature is particularly valuable in complex SDN environments, where traditional fixed function switches may not meet dynamic networking demands [26]. Furthermore, P4's protocol-independent nature ensures that it can accommodate emerging technologies by defining packet processing operations without being restricted to specific protocols [27].

Another significant benefit of P4 lies in its support for robust verification and debugging processes. Tools such as P4b and P4R-Type provide formal frameworks for error checking and verification of P4 programs, reducing deployment risks and enhancing network reliability [28, 29]. Furthermore, P4 enables dynamic adjustments to data plane configurations, improving resource utilization and minimizing control plane overhead [30].

P4 excels in data-plane programmability, but SEC-SDN prioritizes simplicity and performance in centralized control-plane interactions. Incorporating P4 could offset the performance benefits of SEC-SDN's existing real-time security mechanisms. Furthermore, the SEC-SDN framework is designed to scale in various domains, and its current infrastructure satisfies security and scalability goals without requiring advanced programmability of P4. The integration of P4 would also introduce additional verification overhead, complicating deployment and operational processes.

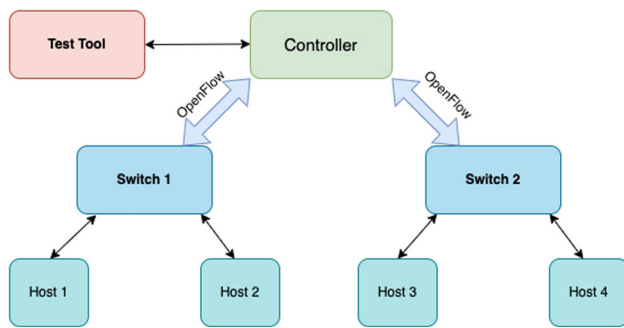


Fig. 4 SDN architecture

In conclusion, the P4 programming language provides substantial benefits in enabling flexible and programmable SDN data planes, making it a key tool in modern network innovations. However, its exclusion from the SEC-SDN framework is justified by the latter's focus on centralized control, simplicity, and specific security requirements. Future SEC-SDN adaptations could explore the potential of P4 in scenarios that require higher data-plane customization or protocol independence.

4 Proposed SEC-SDN framework

In this study, we operate on the assumption that a malicious actor has the capability to compromise either a host or an SDN switch in order to inundate traffic towards either another host or the controller. In the event that a host or switch is infiltrated by malware, it can execute either active or passive attacks. In a passive attack scenario, the infected entity behaves like a typical device, except for discreetly collecting information. In contrast, in an active attack scenario, it deviates from normal behavior to disrupt the network, for example, by flooding attacks.

In the context of a Denial of Service (DoS) attack, a compromised host could potentially inundate packets toward the switch, impeding its performance and generating false requests to the controller. Alternatively, a compromised switch could directly assault the controller, depleting its resources and rendering it unavailable for regular requests. We propose SEC-SDN framework to protect against DDoS attacks.

While OpenFlow offers both proactive and reactive modes, our focus here is solely on reactive applications.

For instance, consider data transmission from *Host1* to *Host4* in Fig. 4. When *Host1* sends a packet to *switch1*, the switch checks its flow table to route the packet. If there are no existing rules for this packet, *switch1* creates a request encapsulating the packet's information and forwards it to the controller. This request, known as a *Packet – In* message, prompts the controller to review its policies and respond with

a *Packet – Out* message back to the switch (steps 2 and 3 in Fig. 5). The *Packet – Out* message provides instructions for handling the new packet, typically by adding a new flow entry to the switch's flow table. If the SDN application's routing policy mimics traditional network operations, the controller might instruct the switch to broadcast the packet to locate the destination. The flow table of each switch is updated with new entries as they respond to the broadcast. Once *switch1* gets a response from *Host4* through *switch2*, it routes the packet using the correct port per its flow table (steps 4 and 5 in Fig. 5). The initial packet undergoes five steps to reach its destination, but subsequent packets from *Host1* to *Host4* follow just three steps (steps 1, 4, and 5), as they are guided by the flow entry already in the switch. This mechanism allows the client host to facilitate interaction between the control and data planes, even though it can't directly communicate with the control plane. In scenarios involving multiple switches between the source and destination, a new packet triggers several *Packet – In* messages across the network, as each intermediary switch contacts the controller for guidance.

SEC-SDN represents an SDN framework designed to identify flooding DDoS attacks by analyzing the statistics gathered on the controller. It includes fundamental routing functions that handle packets in a manner similar to a traditional router. Our objective is to formulate a model capable of pinpointing the origin of an attack, whether it originates from a host or a switch, and the mitigation strategy should not excessively consume network resources, such as bandwidth.

Within SEC-SDN, the detection capability primarily concentrates on *Packet – In* messages, assessing both the volume of received *Packet – In* messages and the controller's capacity to process packets.

Notations of the SEC-SDN parameters:

i - represents a controllers' port numbers where $i=1,2,3,...,n$;

j - represents a switch port numbers where $j=1,2,3,...,m$;

T - represents the period for reset of the threshold;

t - represents counters' monitoring period;

t' - represents the compared time interval between the threshold and counters during DDoS;

t_h - represents time before drop command removed on the switch;

t_d - represents the time before the drop command is removed from a specific port;

γ_{ij} - represents the number of received *Packet – In* from port i of the controller and port j of the switch in a t period of time;

α_i - represents controller port i threshold for DDoS detection;

α'_i - represents controller port i threshold for recovery;

β'_i - represents the number of received *Packet – In* on a port in t' under attack;

σ - represents CPU utilization;

ω - represents $\max(Packet - In)$ in t interval when $\sigma\%$ CPU utilization;

λ_i - represents number of $Packet - In$ received in T on port i ;

τ_i - represents threshold candidate on port i .

Prior to implementing SEC-SDN in an SDN environment, administrators need to establish default thresholds or parameters, such as t and ω . These values are contingent upon hardware performance and specific customer requirements. If the network accommodates more traffic, larger values for ω and σ may be suitable. Once the baseline is configured, SEC-SDN has the capability to autonomously adapt the trigger for detecting DDoS attacks based on network statistics. In addition to routing functions, SEC-SDN primarily encompasses threshold update, monitor, detection, and defense functions. The threshold update function sets the threshold for the monitor function, calculated based on data provided by the monitor function. The monitor function gathers statistics and contrasts the real-time counter with the threshold, notifying the threshold update function if everything is normal or the detection function if there are suspicions. The detection function assesses whether a Distributed Denial of Service attack is underway and activates the defense function to obstruct the attack traffic. Once the DDoS attack concludes, the defense function ceases mitigation and resets the threshold.

4.1 Threshold function

SEC-SDN employs a variety of counters to differentiate between typical and questionable requests. To tell apart legitimate traffic spikes from attacks, SEC-SDN adjusts thresholds based on historical data. For example, if a switch port shows an unusual increase in requests but still within the normal range, SEC-SDN modifies the port's threshold. Consequently, this traffic isn't labeled as an attack. In a regular state, this system tracks the number of Packet-In messages each controller port i receives, using two metrics: β_i and α_i . β_i is compared with α_i at every interval t , while α_i helps establish a potential threshold τ_i at every interval T . These counters aren't used during DDoS attacks, as data collected during such attacks don't accurately represent normal network activity. During a DDoS attack, the system counts $Packet$ messages for each controller port i using a different metric β'_i , and compares this with α'_i at every altered interval t' to determine if the attack has ceased. In this scenario, while malicious requests aren't processed, the controller continues to monitor the southbound interface connecting the switch and the controller.

After the attack concludes, the threshold α_i is reset to its default value, initiating the threshold update process anew. This is because the data gathered during DoS attacks are not reliable for future reference.

4.2 Monitor function

The Monitor function is SEC-SDN's central component, consistently gathering network statistics and comparing them against established thresholds. This comparison helps determine if other modules need to be engaged. SEC-SDN specifically monitors each port on switches directly linked to the controller. The Monitor function tracks the count of $Packet$ messages received over a specific time frame to assess the network's status. If the network is stable with no unusual traffic surges, the function continues to operate without activating additional SEC-SDN modules. However, if it detects abnormal traffic, either the Threshold Update or the DDoS Detect function is triggered. To facilitate this, a listening session is established on the connection between the switch and controller, allowing for the analysis of all received Packet-In messages. These messages also provide source details like switch ID and port ID, which are crucial for identifying devices in the network and localizing DDoS attacks using the DDoS Detect function. In the monitoring phase, SEC-SDN evaluates and records the number of $Packet$ messages received every t seconds through each specific controller port ($i = 1, 2 \dots n$) and switch port ($j = 1, 2 \dots m$), saving this data in the counter γ_{ij} as follows:

$$\gamma_{ij}(t) = \sum Packet_{ij}(t) \quad (1)$$

The total number of $Packet-in$ messages received from a single port of the controller, denoted as β_i , is compared with its corresponding threshold α_i at regular intervals, every t seconds. This comparison is crucial for determining the current status of the network, whether it is normal or exhibiting signs of abnormal activity. The value of β_i is calculated as follows:

$$\beta_i(t) = \sum_{j=1}^m \gamma_{ij}(t), \quad (2)$$

The benchmark of DDoS attack is established by a numerical value ω , which signifies the performance capacity of the controller. ω is determined based on the number of $Packet$ messages that need to be received within a time interval t to cause the CPU utilization of the controller to reach a specified level, $\sigma\%$. The calculation of ω is represented as follows:

$$\omega(t) = \sum_{i=1}^n \beta_i(t), \quad (3)$$

It's important to predefine the value of ω before deploying SEC-SDN, as it relies on the specific software characteristics and hardware resources of the controller. If the value of a single β_i exceeds its corresponding α_i , but the total sum

of β values remains below ω , as shown below, SEC-SDN interprets this situation as burst traffic rather than a DDoS attack and proceeds only with the threshold update. This distinction is crucial for the accurate and efficient functioning of the system.

$$\text{If } \beta_i(t) > \alpha_i(t) \\ \text{and } \sum_{i=1}^n \beta_i(t) \leq \omega(t)$$

then $\alpha_i(t) = \tau_i(T)$, (4)

Thus, α_i will be updated to match τ_i , accommodating the acceptable variations within the network. The threshold update function revises α_i using historical data, and its refresh interval, T , is significantly longer than the monitoring period t . Every T seconds, a new list of candidate thresholds τ_i is generated for each controller port, derived from α_i as per Eq. (2). The default value of α_i is calculated by equally dividing ω by the number of controller ports n , as shown in the following formula:

$$\text{default } \alpha_i(t) = \frac{\omega(t)}{n}, \quad (5)$$

Therefore, α_i represents the maximum number of packets allowed on a specific controller port within each interval t . It serves as a crucial boundary distinguishing between normal and abnormal operations. A straightforward equation for determining the candidate threshold τ_i is as follows:

$$\tau_i(T) = \left[\omega(t) * \frac{\lambda_i(T)}{\sum_i^n \lambda_i(T)} \right], \quad (6)$$

When the Monitor function detects that the number of requests from a switch exceeds the permitted limit, it triggers the DDoS Detect function for a more in-depth analysis and inspection.

4.3 DDoS detection function

The primary role of the DDoS Detect function in SEC-SDN is to identify and locate DDoS attacks within the network, enabling the implementation of appropriate countermeasures to prevent the controller from processing illegitimate requests. The essence of a DDoS attack is the depletion of network resources, characterized either by CPU/bandwidth utilization reaching a critical threshold or by an excessive number of requests.

In real-world testing, it was observed that switches are often the first to be overwhelmed in a DDoS attack targeting the control plane. Therefore, the resource consumption on the switch is used as a threshold indicator for a DDoS

attack. However, due to the inability to monitor switch CPU utilization from the controller and the sufficiency of Southbound Interface (SBI) bandwidth to handle Packet-In requests, the strategy involves determining the maximum number of requests a switch can process within a short period. This value is then translated into the threshold ω .

To define a DDoS attack in this context, if the value of a single β_i exceeds α_i , SEC-SDN initiates a check on the overall β . If the total β surpasses ω within time t , it is classified as a DDoS attack. The process is described as follows.

$$\text{If } \beta_i(t) > \alpha_i(t) \text{ and } \sum_{i=1}^n \beta_i(t) > \omega(t), \text{ then DDoS ON, } (7)$$

Given that the threshold ω is based on the CPU utilization of the controller, SEC-SDN is designed to detect DDoS attacks even if an attacker employs a two-stage strategy: initially generating normal burst traffic to adjust the α_i threshold, followed by launching DDoS attacks to create seemingly acceptable burst traffic at each port. SEC-SDN can still identify such attacks as long as they result in CPU overload.

When a DDoS attack is detected on a controller port, the SEC-SDN approach involves determining whether the attack originates from a switch or a host. This is done by sending a Packet-Out message to the suspicious switch. The Packet-Out message includes a flow rule modification with low priority, instructing the switch to drop all mismatched packets. This new flow entry, once installed on the switch, prevents it from generating Packet-In messages for new packets. Importantly, this operation has a lower priority than existing flow entries, ensuring that authorized users can still transmit data through current network paths.

The Packet-Out message also incorporates a hard-timeout feature (defined in OpenFlow), which automatically removes this rule after a predefined duration, t_h seconds, regardless of its activity. This temporary rule allows the controller to ignore new requests for a brief period. If the controller continues to receive Packet-In messages from the switch within this time frame, SEC-SDN concludes that the switch is compromised, as it is not adhering to the flow table rules and is likely sending fabricated packets. Conversely, if the influx of Packet-In messages ceases, it suggests that a host is the source of the attack, not the switch.

SEC-SDN then utilizes the γ_{ij} values to identify the attacker, pinpointing the switch port with the highest γ_{ij} value as the compromised source. This approach enhances SEC-SDN efficiency by avoiding the need to block unknown packets indiscriminately. Once the source of the attack is identified, SEC-SDN activates its DDoS defense module to block malicious access. This targeted response allows SEC-SDN to effectively mitigate and manage the impact of DDoS attacks on the network.

4.4 DDoS defense function

During the DoS defense phase, SEC-SDN implements distinct countermeasures for attacks originating from either a host or a switch, based on the findings from the DoS Detect function.

Attack from a Host:

1. Drop Policy: SEC-SDN inserts a drop policy into the switch specifically targeting the offending host's switch port. This policy discards strange packets originating from that port.
2. Flow Modification: This policy differs from the detection phase in two key aspects:
 - It targets only the specific port on the switch linked to the hostile host.
 - An idle timeout is set, so that if no packets trigger this rule for a consecutive duration of t_d seconds, the drop policy is automatically removed.

Attack from a switch:

1. Connection Severance: If the switch itself is the source of the attack and is thus unmanageable, an obvious response is to cut off its connection to the controller. However, this poses a challenge in determining when to safely reestablish the connection.
2. Monitoring without Processing: SEC-SDN continues to count Packet-In messages from the infected switch but does not process them. This information is stored in β'_i (as per Eq. 7), conserving controller resources while monitoring the situation.
3. Threshold comparison: β'_i is periodically compared with a default threshold α'_i every t' seconds to assess whether the attack has stopped.
4. Restoration: If β'_i falls below or equals α'_i , SEC-SDN considers the attack over and initiates a threshold update, resetting α_i to default values. This “starting from scratch” approach ensures that the previous threshold, possibly compromised by the DDoS attack, does not influence the new settings. Both methods autonomously cease once the network returns to a normal state, and thresholds are reset post-attack. This design allows SEC-SDN to not only manage DDoS attacks effectively but also autonomously restore network configurations to their pre-attack state. In essence, SEC-SDN equips the network with self-healing capabilities in the face of DoS attacks.

5 Enhancing the SEC-SDN framework with game theoretic strategies

To integrate a game-theoretic approach into the cybersecurity framework using SDN, we can leverage mathematical formulations and strategies in a game-theoretic approach to network security using honeypots [17]. The integration focuses on enhancing the SEC-SDN framework's ability to anticipate and mitigate cybersecurity threats, specifically DDoS attacks, by incorporating a decision-making model that adapts based on the actions of potential attackers.

The SEC-SDN framework, designed to protect against DDoS attacks within SDN environments, could be significantly enhanced by incorporating game-theoretic strategies to dynamically adjust security measures in response to attacker behavior. This integration involves the following steps:

1. Modeling Attacker and Defender Strategies: Utilize the Course of Action Stackelberg Game (CoASG) from the game-theoretic approach, where the defender (the SEC-SDN framework) acts as the leader, and the attacker as the follower. The framework must predict potential attack strategies and adjust its defensive posture accordingly.
2. Implementing the SE in SEC-SDN: Integrate the concept of SE to determine the optimal defense strategy considering the attacker's best response to the defender's actions. This involves adjusting the SEC-SDN's parameters, such as thresholds for DDoS detection and the selection of mitigation strategies, based on anticipated attacker behaviors.
3. Dynamic Threshold Adjustment: Using the game-theoretic approach to dynamically adjust thresholds to identify DDoS attacks, similar to the process described in the SEC-SDN framework. The thresholds (e.g., for Packet-In messages or CPU utilization) would not be static but would adapt based on a game-theoretic analysis of ongoing network conditions and potential attacker strategies.
4. Optimization of Defense Strategies: Apply the backward induction algorithm to find the SE as a method to systematically explore and optimize defense strategies within the SEC-SDN framework. This involves evaluating various defensive actions (e.g. blocking suspicious IP addresses, adjusting flow rules) to identify those that minimize potential damage from DDoS attacks while considering the cost of false positives and resource consumption.
5. Simulation and Evaluation: Before deployment, simulate various attack scenarios against the enhanced SEC-SDN framework to evaluate its effectiveness in real-time DDoS mitigation. Use game-theoretic metrics to assess the framework's performance, such as the success rate in

thwarting attacks, resource efficiency, and the accuracy of attack detection.

By integrating these game-theoretic strategies into the SEC-SDN framework, the system can become more proactive and adaptive in its defense mechanisms. This approach not only enhances the resilience of the network against DDoS attacks but also provides a structured methodology for continuously evolving the network defense strategies based on the changing dynamics of cyber threats.

To mathematically correlate the game theory approach described in the first article [17] with the SEC-SDN framework outlined in the second article [3], we can propose a function that aligns the strategic defense mechanisms involving honeypots with the dynamic security configurations enabled by SDN. This integration can be modeled by incorporating the concepts of SE and Course of Action Stack-berg Game (CoASG) into the SDN security framework.

Given that $G^d(a, d)$ represents the defender's payoff in a game theoretic model where a and d are the attacker's and defender's actions respectively, and considering SDN's ability to reconfigure network resources dynamically, we can define a function that captures the overall security level S of an SDN-enabled network as a function of the defender's payoff and the network's configuration state C . The configuration state C represents the set of all possible network configurations enabled by SDN, which can be adjusted in response to detected threats.

Let $S(C, d)$ be the security level of the network, which is a function of the network configuration state C and the actions of the defender d . This can be expressed as follows.

$$S(C, d) = \max_{d \in D} G_d(\text{resp}_0(d), d) \cdot f(C) \quad (8)$$

where

- $G_d(\text{resp}_0(d), d)$ is the defender's payoff in the SE with the lowest cost, as defined in the game-theoretic model,
- $\text{resp}_0(d)$ is the best response function of the defender considering the attacker's actions,
- $f(C)$ is a function that quantifies the effectiveness of the network configuration state C in enhancing the network's security posture,
- D is the set of all possible defender's actions within the SDN framework.

The function $f(C)$ reflects the adaptability of the SDN framework to respond to threats by adjusting its configurations. It could be further defined based on specific metrics such as the reduction in the attack surface, improved detection rates, or the reduced impact of successful attacks, which are directly influenced by the way effectively the SDN

controller reconfigures the network in response to ongoing attacks.

This model underscores the synergy between game-theoretic defense strategies and SDN's dynamic reconfiguration capabilities, offering a comprehensive approach to network security that leverages the predictive power of game theory with the flexibility of SDN.

6 Implementing and evaluating the SEC-SDN framework

6.1 Modeling attacker and defender strategy

We conceptualize the DDoS attack as a dynamic game between the attacker and the network administrator. The attacker's objective is to compromise critical infrastructure by overwhelming it with a massive influx of traffic, utilizing bots that are positioned both within and outside the targeted network. While some of these bots might be identified and intercepted by an Intrusion Detection System through signature matching, it is acknowledged that contemporary DDoS botnets operate with a high degree of stealth. We proposed a game-theoretic model designed to comprehensively expose the botnet and implement rate-limiting on traffic originating from these malicious entities. This model employs the principle of reward and punishment, a mechanism commonly used in game theory to foster cooperation among entities, adapted here to encourage compliance and deter malicious activities. By applying punitive measures against agents or users that demonstrate harmful behavior, the model aims to maintain a balanced and secure network environment. Our system is conceptualized as a dynamic multi-player game, where a single network administrator (represented by the SDN Controller in our scenario) contends against multiple players, including attackers. The administrator's strategies are implemented through the deployment of OpenFlow rules.

The extensive form game tree for this scenario is depicted next from the payoff matrix calculated in [17] and expressed in Figure 5.

6.2 Optimizing defense strategies in SEC-SDN

P_1 denotes the attacker, while P_2 represents the network administrator and B represents bandwidth. By referring to the payoff matrix, we can ascertain the average bandwidth at the end of period $t = 2$, which is $\frac{B}{2}$, given that the user cooperates at $t = \{0, 1\}$. In the event that the user P_1 acts maliciously, they would initially gain bandwidth

$$\frac{3B}{4} \text{ at } t = 1 \quad (9)$$

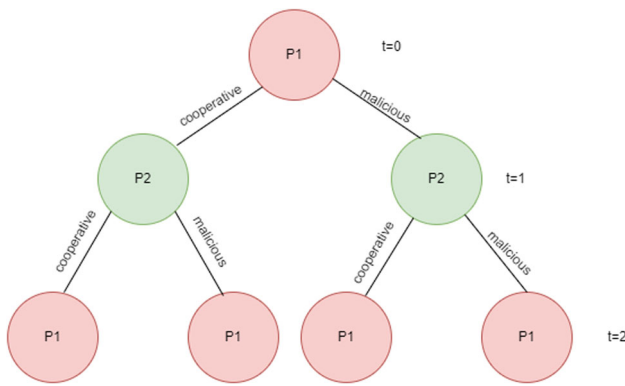


Fig. 5 Dynamic game

However, in response, the administrator P_2 would impose a punishment at $t = 1$, resulting in a reduced bandwidth (B_r)

$$B_r = \frac{B}{5} \quad (10)$$

for the following period. This leads to an average B_a of

$$B_a = \frac{1}{2} \times \left(\frac{3B}{4} + \frac{B}{5} \right) = 0.475B \quad (11)$$

which is lower than $0.75B$ that would have been achieved if P_1 had maintained cooperative behavior. The countermeasures taken by the administrator in response to an attack are colored red along the path. In the long term, the attacker is incentivized to adopt normal behavior (refraining malicious traffic) if a rate-limiting mechanism is used, as explained in Algorithm 1.

Algorithm 1 Dynamic Game Between Network Administrator and Attackers

```

1: Initialize game tree with players  $P_1$  (attacker) and  $P_2$  (administrator)
2:  $B \leftarrow$  Initial bandwidth
3: for  $t = 0$  to End do
4:   if  $P_1$  cooperates then
5:      $B_i \leftarrow \frac{3B}{4}$   $\triangleright$  Bandwidth allocated if  $P_1$  cooperates at  $t = 1$ 
6:     Update  $P_2$ 's strategy to maintain normal flow
7:   else  $\triangleright P_1$  defects
8:      $B_i \leftarrow \frac{3B}{4}$  at  $t = 1$ 
9:      $P_2$  punishes  $P_1$  at  $t = 1$ , resulting in  $B_r = \frac{B}{5}$ 
10:     $B_a \leftarrow \frac{1}{2} \times \left( \frac{3B}{4} + \frac{B}{5} \right)$ 
11:   end if
12:   if punishment period is over then
13:     Reset  $B_i$  to default rate for cooperative behavior
14:   end if
15: end for
16: Evaluate long-term outcome for  $P_1$  and adjust strategy accordingly

```

The system architecture, which is built on a Ryu-based SDN platform, utilizes southbound APIs to communicate with elements in the data plane. The OpenFlow compatible

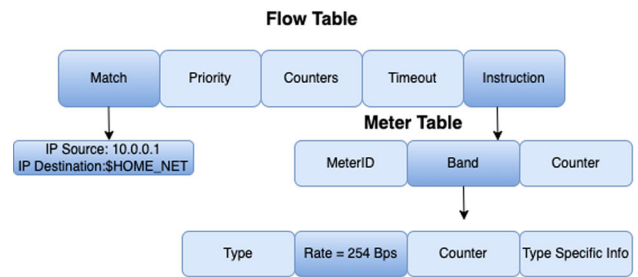


Fig. 6 Traffic limit in SDN

switches facilitate interactions with hosts, both within and external to the network. Certain hosts transmit regular traffic to the switch, whereas others participate in a DDoS botnet. The SDN controller framework comprises multiple components, including a topology manager responsible for network topology adjustments and a network configuration element to maintain the network's current setup.

We employ a signature-based detection system in Snort to gather network traffic and classify it as malicious or benign. This data is relayed to the network service and orchestration layers via northbound REST APIs.

6.3 Implementation of SEC-SDN and bandwidth limiting algorithm

We focus on three specific types of DDoS attacks: SYN flood attack, UDP flood attack, and ICMP flood attack. Upon activation by Snort, the DDoS defense mechanism will prompt an update to the SDN flow table. This update aims to impose a rate limit on traffic from a specific source to a destination within the home network.

Figure 6 illustrates various entries within a specific SDN flow table. The match field is utilized to identify the incoming port and packet headers. In this scenario, a Snort alert is triggered for the IP Address 10.0.0.1, identified as the attacker's IP. The instruction field within the flow table receives an addition. The rate limit, as determined by the algorithm, is configured in the rate subfield of the Band Field within the Meter Table. According to Fig. 6, once the punishment period set by the administrator for this IP address is completed, the Rate sub-field will revert to the default traffic burst rate. The REST API is used to consistently push these updates to the controller.

By default, the meter table is not a mandatory component of a Flow Table. A host that behaves normally will not experience any reduction in bandwidth. However, if the host exhibits malicious behavior and a trigger, such as a Snort IDS alert, activates the meter with ID 1, the bandwidth will be adjusted according to a rate-limiting policy. This adjustment is based on whether the malicious host cooperates or is in a defect in the current and future periods.

6.4 Evaluating SEC-SDN framework

Our evaluation involved conducting two experiments using a network simulator and a Ryu controller on Ubuntu 22.04 OS. The initial experiment implemented the Algorithm 1 to address ICMP flood attacks. In the second experiment, we applied the same algorithm to counter TCP SYN flood and UDP flood attacks within a fat-tree network topology. The selection of different topologies for these experiments was aimed at assessing the universal applicability of our proposed solution.

6.4.1 Phase 1: DDoS attacks with ICMP flood in linear topology

In the initial experiment, we established a linear topology within the Mininet environment, adjusting the number of hosts from 50 to 500. This topology featured a single layer of hosts, each connected to a singular switch, as illustrated in Fig. 4. We developed a Python script to orchestrate an attack, employing multiprocessing to initiate a shell for each host, thereby generating ICMP traffic with large packet sizes directed towards a single host within the network. This traffic was port-mirrored to a dummy port, enabling the Intrusion Detection System (IDS) to detect the ICMP flood DDoS attack signature and subsequently relay this information to the Ryu controller.

The Ryu application for DDoS mitigation methodically lowers the traffic rate by a multiplicative factor of δ until the traffic's long-term average corresponds with normal traffic bursts from a designated host. In this particular study, the damping factor δ was set to 0.8. Unlike approaches that block traffic entirely or apply a static rate limit, this strategy gradually reduces the throughput for attacking hosts, thereby minimizing impact on legitimate user traffic.

The table illustrates the impact of traffic bursts at the target for 100 hosts, recording a traffic rate of 78.12 Mbps in the absence of any attack prevention mechanism to counter DDoS attacks. With activation of the Rate Limit (RL) trigger by the IDS, traffic significantly reduces to 2.56 Mbps, indicating a reduction factor of 30. As the number of attacking hosts increases from 100 to 500, the throughput attributable to the DDoS attack climbs from 78.12 to 456.77 Mbps, evidencing a linear growth in attack traffic. The algorithm adjusts to the surged traffic, reducing the corresponding traffic limit for 500 hosts to 15.27 Mbps. A comparative analysis of attack traffic and rate-limited traffic for 500 hosts reveals a reduction factor of 29. This experiment corroborates the efficacy of a game-theoretic approach in mitigating attacks, serving as a successful countermeasure within a sufficiently extensive network.

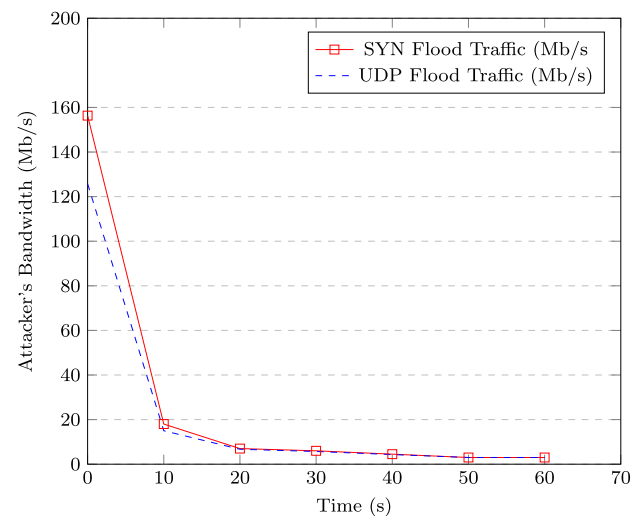


Fig. 7 UDP and TCP flood attack mitigation on fat tree topology

6.4.2 Phase 2: DDoS attacks with TCP/UDP flood in fat tree topology

Many of the cyber attacks targeting organizations are aimed at DNS servers, dispatching extensive volumes of TCP or UDP packets towards the targeted hosts. Additionally, considering the prevalent use of fat-tree topology in data center architectures, we performed experiments to assess our algorithm on a fat-tree topology configured in Mininet, characterized by a depth and fanout both set to 3. In this particular study, we applied a damping factor δ of 0.9.

In this experiment, the SDN controller predefined the acceptable limit for TCP and UDP traffic as 3.0 Mbps. A TCP SYN flood DDoS attack was launched on a network topology consisting of 64 hosts. The traffic reduction, subsequent to triggering the rate limiting algorithm as an attack countermeasure based on IDS alerts, is depicted in red in Fig. 8. Initially, DDoS traffic surged to 156.33 Mbps, starkly surpassing the set permissible threshold. Upon the SDN controller's response to counteract the attack, the traffic levels were reduced to 18.11 Mbps at $t = 10$ s. The traffic intensity eventually stabilized at 3 Mbps by $t = 50$ s, which is nearly equivalent to the normal traffic rate authorized for this network (Fig. 7).

In a UDP flood attack scenario, the initial traffic flow starts at approximately 125.77 Mbps. The application of the rate-limiting algorithm reduces this traffic rate to 15 Mbps at $t = 10$ s. A further reduction is observed by $t = 40$ s, where the traffic rate decreases to 4.32 Mbps. At $t = 60$ seconds, the traffic burst stabilizes at 3 Mbps, after which the algorithm discontinues further rate-limiting measures. At this juncture, the Intrusion Detection System (IDS) remains on alert for additional intrusion signals, ready to notify the controller if attacking hosts persist in sending malicious traffic.

Table 2 SEC-SDN parameters algorithm used in simulation

Attacking hosts	ICMP flood traffic (Mb/s)	ICMP traffic post rate limit (Mb/s)
50	38.22	1.23
100	78.12	2.56
200	161.84	5.43
300	240.01	8.02
400	319.21	10.23
500	456.77	15.27

This experiment illustrates that the algorithm can effectively mitigate TCP and UDP-based DDoS attacks on significantly large networks in less than one minute.

7 Testing the SEC-SDN framework in an openstack cloud architecture

7.1 Integration and setup

The integration of the SEC-SDN framework into OpenStack is designed to enhance the cloud's ability to resist DDoS attacks by leveraging advanced, software-defined network security measures. The primary goal is to establish seamless communication between the SEC-SDN framework and OpenStack's Neutron service, enabling effective monitoring and management of virtual network traffic. This integration aims to facilitate real-time security policy management and dynamic threat response capabilities within the cloud environment.

The integration requires an operational OpenStack installation updated to the latest version, which at the time of testing is *Caracal, Openstack 2024.1* that supports the Neutron API. The SEC-SDN framework is compatible with the OpenFlow protocol and should integrate smoothly with OpenStack Neutron for network management. Adequate hardware resources are essential to support the additional load from the SEC-SDN framework, especially during DDoS attack simulations. The network infrastructure must be capable of handling high traffic volumes and provide isolation for testing scenarios.

We utilized an existing OpenStack installation configured with standard compute, storage, and network services. The OpenStack version used supports the latest Neutron API functionalities. The SEC-SDN framework was installed on a dedicated virtual machine within the cloud environment. This machine was configured to communicate with OpenStack Neutron and the underlying physical network infrastructure. Several tenant networks were created in OpenStack, each configured with unique security requirements and traffic profiles to simulate real-world usage scenarios. Prior to the SEC-SDN implementation, we recorded the baseline perfor-

mance and security metrics of the OpenStack cloud under normal operation and during simulated DDoS attacks. Using traffic generation tools, we simulate various types of DDoS attacks, including SYN Flood, ICMP Flood, and UDP Flood. The attacks targeted virtual machine instances and network components within the OpenStack environment. The SEC-SDN framework was activated to detect and mitigate attacks. Metrics such as attack detection time, mitigation effectiveness, and impact on legitimate traffic were measured. The framework's ability to automatically adjust security policies in real-time in response to detected threats was evaluated. This included the dynamic configuration of network flow rules and security policies managed through the OpenStack Neutron service.

7.2 Test environment configuration

The test environment is meticulously crafted to emulate a typical cloud infrastructure involving 100 virtual hosts on the OpenStack platform. Of these, 90 hosts simulate normal cloud operations that run everyday applications and services, while the remaining 10 are configured as attacking hosts. These attacking hosts are tasked with simulating various Distributed Denial of Service (DDoS) attacks to test the network's defensive mechanisms. In order to test in the cloud the SEC-SDN framework we installed fewer VMs in a 1:50 ratio in comparison with the simulation in Mininet and we compared the results with those from Table 2.

The regular hosts are configured identically to support standard cloud operations, each equipped with 1 vCPUs and 2 GB of RAM, and are distributed across multiple tenant networks to mimic a real-life multi-tenant scenario. In contrast, the attacking hosts are also set up with similar hardware configurations to blend in with the normal traffic, yet are installed with tools designed to generate malicious traffic, such as SYN Floods, ICMP Floods, and UDP Floods. This setup is intended to create a realistic environment where attacks emerge from within the network, mimicking insider threats or compromised machines.

Network management is handled by OpenStack's Neutron service, which facilitates the creation of isolated networks using a combination of VLANs and VXLANs for effective

segmentation. This network setup is critical for managing the diverse traffic flows and ensuring that the attacking traffic does not overwhelm the entire network. The virtualization layer is powered by KVM, managed through OpenStack's Nova service, with SDN controllers integrated for dynamic network routing and security policy management.

For intrusion detection, Snort is deployed as the IDS to monitor network traffic for signs of intrusion and abnormal activities, specifically from the attacking hosts. Snort's capabilities are utilized to distinguish between normal high-traffic loads and potential security threats, ensuring that real attacks are identified promptly without producing false alarms. Alongside Snort, comprehensive monitoring and logging of all network traffic are conducted to track performance metrics such as traffic volume, packet loss, and response times, which are crucial for analyzing the impact of attacks and the effectiveness of deployed mitigation strategies.

DDoS attacks are simulated at controlled intervals, and attacks target specific network resources to assess the resilience of the infrastructure. The SEC-SDN framework response to these attacks is critically evaluated, focusing on the speed and effectiveness of the detection and mitigation processes facilitated by Snort. Additionally, the framework's ability to sustain normal operations for the non-attacking hosts during these attacks is tested, ensuring that the system can segregate and neutralize threats without disrupting regular activities.

Snort is deployed on a dedicated virtual machine within the OpenStack environment. This VM configured with 8 GB of RAM, 4vCPU and 100 GB of HDD will act as the Snort server, analyzing traffic that flows through the network. The VM should be provisioned with sufficient resources (CPU, memory, and disk space) to handle the expected network traffic and log data, but only for the testing purposes the configuration will be sufficient. Once Snort was installed, it was configured to recognize the network architecture and the types of traffic that should be monitored. This includes setting up the proper rules that define what constitutes suspicious activities. Snort rules need to be continually updated to adapt to new threats.

To monitor network traffic flowing through the OpenStack SDN, Snort needs to receive a copy of this traffic. This was achieved by configuring port mirroring (also known as SPAN) on the virtual switches within the OpenStack environment. Neutron was configured to redirect a copy of the network traffic from each virtual switch to the Snort VM.

Snort processes the mirrored traffic in real-time, using its rule set to analyze packets and look for matches to known attack signatures or anomaly traffic patterns. When an attack or anomaly is detected, Snort generates alerts. These alerts can be configured to trigger automatic responses from the SDN controller, such as adjusting firewall rules or modifying access control lists to mitigate the threat immediately.

To automate the response further, Snort is integrated with the SEC-SDN used in the OpenStack environment. SEC-SDN will automate the deployment of additional security measures, such as launching new firewall instances or reconfiguring network settings based on alerts from Snort. This level of automation helps in maintaining security in dynamic cloud environments where network configurations and workloads can change frequently.

7.3 Testing methodology

The testing methodology for evaluating the SEC-SDN framework within the OpenStack environment involved a combination of automated testing tools, custom scripts, and manual oversight to comprehensively assess the effectiveness and efficiency of security measures against DDoS attacks. This approach was designed to ensure a thorough assessment of the network security features and the response mechanisms of the framework.

Prior to the commencement of attack simulations, baseline performance metrics such as response times, throughput, and resource utilization (CPU, memory, and network bandwidth) were established. These metrics were essential for comparing the system performance under normal and attack conditions. Wireshark was employed to monitor and collect these baseline metrics. Its comprehensive data visualization capabilities helped to establish a clear performance benchmark for later comparison.

Various types of DDoS attacks were simulated to test the robustness of the SEC-SDN framework. These simulations aimed to mimic realistic attack scenarios that could potentially disrupt the network infrastructure. During DDoS attack simulations, the SEC-SDN framework was automatically triggered to activate predefined security policies designed to mitigate the impact of the attacks. This phase tested the responsiveness and effectiveness of the framework in a live attack scenario. The SEC-SDN framework successfully identified and mitigated the influx of malicious traffic, demonstrating robust detection capabilities and prompt activation of mitigation strategies.

The SEC-SDN framework significantly enhances the security of cloud networks by integrating advanced detection algorithms and dynamic policy management capabilities within the OpenStack environment. The SEC-SDN framework was deployed within an OpenStack environment, leveraging Neutron for network configuration and management. By integrating with Snort IDS, the framework could analyze traffic for malicious patterns and initiate automatic security updates across the network.

Upon detection of potential threats by Snort, the SEC-SDN framework dynamically updated the security policies in Neutron. The process begins with the Snort IDS monitoring network traffic for unusual or malicious patterns. When a

potential threat is detected, such as an apparent DDoS attack or scanning activity from a specific IP address or port, Snort generates alerts based on its configured rule sets. These alerts are then communicated to the SEC-SDN controller, which acts as the central decision-making body within the software-defined network. The controller evaluates the alert to confirm the threat level and decides whether an IP blocking rule or port blocking rule needs to be applied.

Once a decision is made to block an IP address or port, the SEC-SDN controller sends a command to Neutron, OpenStack's networking service, to update the network access policies. This command includes details of the specific IP addresses or ports to be blocked. Neutron implements these commands by updating the security group rules associated with the relevant network or subnet. The security groups in OpenStack act like virtual firewalls that define which traffic can enter or leave network interfaces (ports) attached to OpenStack instances (virtual machines). The specific rules to block IP addresses are enforced at the virtual switch level within the OpenStack environment. This can involve dropping all incoming and outgoing packets to and from the identified malicious IPs, effectively isolating them from communicating with any resources within the cloud environment. The SEC-SDN framework automates this workflow from detection to action. By integrating Snort with the SDN controller and Neutron, the response to threats is rapid and precise, minimizing the window of exposure to an attack. The system allows for dynamic adjustments to the blocklist as new threats are identified or as false positives are corrected. This flexibility is crucial to adapting to evolving security landscapes where threats can change rapidly.

Continuous monitoring and logging are essential components of this process. They provide feedback on the effectiveness of IP blocking and help identify any adjustments needed to the security configurations or rules. All actions taken by the system, including the specific IP addresses blocked and the time of action, are logged for audit purposes. These logs are crucial for compliance with security policies and regulations.

IP blocking is a fundamental component of maintaining network security, particularly in environments susceptible to external attacks. In the context of the SEC-SDN framework, this process not only prevents identified threats from causing harm, but also improves the overall resilience of the cloud infrastructure by enabling proactive and automated security management. We used for traffic throttling iPerf which simplifies the process of sending large numbers UDP, or TCP requests to a target, which can be a server or an entire network. This tool's ability to quickly generate massive traffic volumes can saturate network resources in an SDN environment, where a centralized controller manages network decisions. Such an attack can slow down or completely block

legitimate network traffic, leading to potential service downtimes.

When tools like iPerf initiate traffic patterns that are potentially harmful, Snort detects these anomalies and alerts the SDN controller. In response to these alerts, SEC-SDN dynamically implements traffic throttling measures specifically targeting the IP addresses from which malicious traffic originates. By limiting the bandwidth available to these hosts, SEC-SDN effectively reduces the volume of data they can send, thereby mitigating the impact of the attack on the network. This not only preserves bandwidth for legitimate traffic but also prevents the network from being overwhelmed. Additionally, SEC-SDN employs policy-based network segmentation to enhance security. This strategy isolates critical network resources from less critical ones, confining the spread of any attack within the network and protecting vital data and services. This segmentation can be dynamically adjusted based on ongoing threat evaluations, which allows for flexible and responsive adaptation to new or escalating threats. Furthermore, the SEC-SDN framework is designed to adapt to evolving network conditions and threats through real-time updates to its security policies. This capability ensures that the network's defenses remain effective against new and emerging threats, including sophisticated zero-day exploits.

Overall, SEC-SDN not only counters the disruptive capabilities of tools like iPerf and others but also enhances the overall resilience and efficiency of SDN environments. This ensures that networks can maintain high levels of performance and availability, even under potential DDoS attack scenarios.

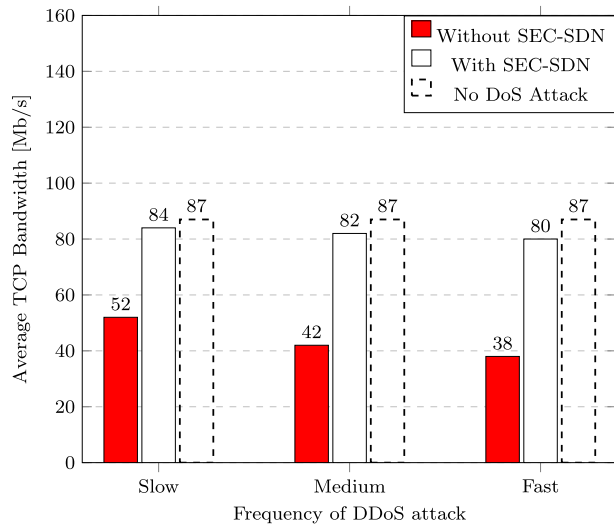
7.4 Effects of DDoS attacks in the SEC-SDN-based cloud environment

To evaluate the effectiveness of SEC-SDN, we tested the network performance under DoS attacks with and without the implementation of SEC-SDN. The sender host continuously transmitted TCP datagrams to the receiver host, while the attacker launched DoS attacks at varying frequencies. The attack frequency was varied by adjusting the *time-to-transmit* parameter ('-t') in iPerf for UDP traffic. The '-t' parameter specifies the duration for packet transmission to a particular destination, thereby controlling how frequently the destination is updated.

As a new destination triggers a Packet-In request, it enables control over the DDoS attack frequency. The bandwidth for UDP traffic is set to 10 Mbps. An example of a command for a slow-frequency attack is "iperf -c 10.0.1.100 -u -t 1 -b 10M". In our experiments, we analyze three types of attack frequencies: slow, medium, and fast, to simulate the DoS attack against SDN. Table 3 presents the definitions of these three types together with the

Table 3 DDoS attacks frequency

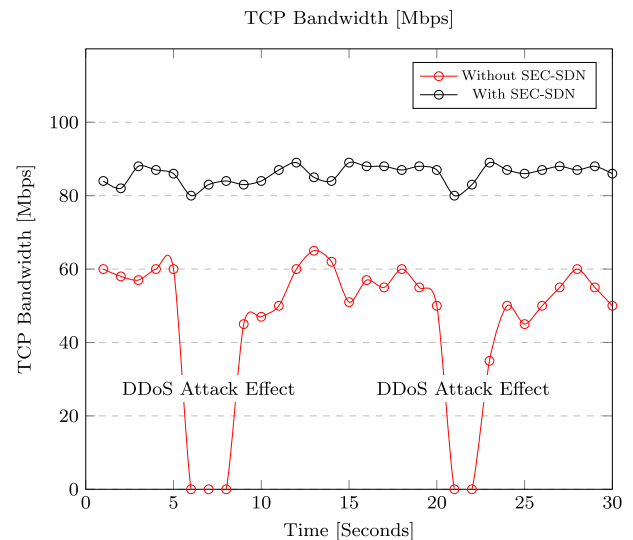
Frequency	Transmission time (seconds)	Number of Packet-In (messages per second)
Slow	1	123
Medium	0.1	256
Fast	0.001	543

**Fig. 8** TCP stream bandwidth under DDoS attacks in SEC-SDN cloud

corresponding number of messages per frequency. The transmission time is defined by the value of ' τ ' in iPerf, and the count of Packet-In messages is the average of 10 trials, measured at the controller using Wireshark.

To assess the available bandwidth under DDoS attacks both in the presence and absence of SEC-SDN in our experiment, we employ iPerf to facilitate TCP traffic between the sender and receiver, utilizing iPerf's ability to generate TCP flows at the maximum possible bandwidth. We also established a threshold at 100 to enable SEC-SDN's mitigation response to slow, medium, and fast attack intensities. Figure 8 illustrates the impact of DoS attacks on the TCP stream bandwidth with the SEC-SDN algorithm engaged compared to without it. The data, representing the result of 10 iterations along with the 95% confidence intervals, enumerate the TCP bandwidth measurements under various DoS attack conditions, both with and without SEC-SDN's involvement. In the absence of any attack, the receiver's TCP bandwidth is approximately 87 Mbps. However, this bandwidth experiences a reduction as the frequency of attacks increases when SEC-SDN is not applied.

With the implementation of the SEC-SDN algorithm, observations indicate that the TCP bandwidth does not decrease and closely approximates the scenario where no attack occurs, maintaining a bandwidth performance between 96.5 and 91.95%. This substantiates the ability of

**Fig. 9** Instant available TCP bandwidth under fast DDoS attacks in SEC-SDN cloud

SEC-SDN to accurately detect and counteract DoS attacks. Moreover, validation through Wireshark confirms that the controller ceases to receive *Packet-In* messages from the malicious host.

Figure 9 depicts the impacts of DDoS attacks on the instantaneous TCP bandwidth. As the attack intensity escalates, a corresponding increase is observed in the instantaneous TCP bandwidth degradation. Specifically in the scenario of a fast attack expressed in Fig. 9, it is noted from the red line in the figures that the TCP bandwidth periodically suffers downtimes when SEC-SDN is not implemented. These downtime periods, indicated with black arrows, are attributed to buffer overflow in the switch. This overflow occurs because malicious packets deplete the buffer capacity, forcing the switch to discard all packets and undergo a reset.

The black line shows that the implementation of SEC-SDN stabilizes the instantaneous TCP bandwidth, maintaining a consistent and steady flow.

The flagged data from SEC-SDN is then fed into Snort configured with a comprehensive set of rules for intrusion detection, which processes the received data. It matches the incoming IP addresses with its rule set to determine if they are known sources of malicious activities or match patterns of known attack vectors. If Snort confirms that the flagged IPs are indeed sources of malicious traffic, it generates alerts and can trigger automated responses. These responses might

include blocking the IP addresses, rerouting traffic, or implementing additional security measures to mitigate the threat.

Information about the incident, including the effectiveness of the mitigative actions, is fed back into the SEC-SDN system. This helps in refining the detection algorithms and updating the security policies in the SDN controller, enhancing the system's predictive capabilities and responsiveness to future threats.

By leveraging the dynamic control capabilities of SEC-SDN and the powerful intrusion detection features of Snort, networks can achieve a more proactive and robust defense mechanism against DDoS attacks and other cyber threats, particularly in complex cloud environments. This integration ensures that suspicious activities are not only detected, but also quickly and effectively mitigated, maintaining the security and performance of the network.

7.5 Comparison of SEC-SDN with other frameworks

The SEC-SDN framework's integration with OpenStack allows it to manage and respond to threats dynamically within the cloud infrastructure. This system not only detects threats through Snort, but also automates the response, adjusting network configurations and security policies in real time based on the nature of the detected threat. This contrasts sharply with the frameworks discussed by Dharma and Rino [10], which, while innovative in integrating Telegram for notifications, do not offer integrated cloud management or automated policy adjustments based on threat detection. This novel use of Telegram enables real-time alert management and could potentially facilitate quicker human response to detected threats. However, this system focuses primarily on leveraging communication tools for alert dissemination rather than integrating deeply with network management systems or cloud infrastructure. In contrast, the SEC-SDN framework is deeply integrated with OpenStack, a cloud operating system that manages large pools of compute, storage, and networking resources. This integration enables SEC-SDN not only to detect threats, but also to manage and respond to them dynamically within the cloud environment. The framework automates responses to detected threats by adjusting network configurations and security policies in real time, directly interacting with the cloud infrastructure to mitigate potential damage without human intervention.

Other studies such as those by Dao et al. [12] and Mousavi and St-Hilaire [13] focus on the feasibility of methods and early detection strategies for DDoS attacks within SDN environments. These studies, while insightful, do not provide details on integration into cloud environments or real-time threat management, which are central to the SEC-SDN framework.

SEC-SDN is deeply integrated with OpenStack, allowing it to leverage cloud infrastructure capabilities to manage net-

work traffic and respond to security threats dynamically. This integration facilitates an automated response where security policies are adjusted in real time based on the detected threats. In contrast, the system proposed by Gala et al. [11] introduces the use of autoencoders combined with Snort for intrusion detection. While this approach enhances detection capabilities with machine learning to potentially improve accuracy and reduce false positives, it does not inherently include details on integration with cloud management systems or automated response mechanisms, which are crucial for real-time threat mitigation in dynamic cloud environments.

The hybrid IDS by Gala et al. [11] benefits from the application of autoencoders, a form of neural networks that can detect complex patterns and anomalies in network traffic, which might not be immediately identifiable with traditional rule-based systems like Snort alone. This could theoretically lead to earlier detection of sophisticated, subtle, or emerging threats that do not match known signatures. However, SEC-SDN, while currently relying on traditional rule-based detection through Snort, is already operational within a dynamic cloud environment, automatically adjusting network settings to mitigate threats as they are detected. This immediate, automated action can be crucial in reducing the impact of attacks, even if the initial detection relies on conventional methods.

The integration of SEC-SDN with OpenStack is particularly advantageous in a cloud setting where network configurations and traffic patterns can be highly variable. The SEC-SDN framework's ability to dynamically update security policies and configure network elements on the fly, based on real-time data from Snort, provides a robust defense mechanism that is both adaptable and scalable. This stands in contrast to the hybrid IDS by Gala et al. [11], where the emphasis is more on the detection phase, potentially lacking the integrated management and automated mitigation response that is critical in cloud environments.

Overall, the SEC-SDN framework offers a more integrated and automated approach to handling network security in cloud environments compared to the more narrowly focused or theoretical approaches discussed in the articles. Its practical application in a live cloud infrastructure, coupled with automated response capabilities based on Snort detection, suggests that it can provide robust protection in dynamic, large-scale environments. This contrasts with the other studies that either focus on specific aspects of intrusion detection or do not integrate into broader cloud management frameworks.

In summary, while each of the discussed systems brings valuable capabilities to network security, the SEC-SDN framework's real-time integration with cloud infrastructure, coupled with its automated threat response mechanisms, positions it as a particularly effective solution for protecting large-scale cloud environments against sophisticated

cyber threats like DDoS attacks. Integrating machine learning could further enhance its capabilities, making it an even more formidable tool in the evolving landscape of network security.

7.6 SEC-SDN limitations and future work

The integration of SDN with game theory and IDS such as Snort presents notable limitations that need to be addressed in future research. These limitations stem from the complexity of combining these technologies, which often leads to challenges in computational efficiency, scalability, interoperability, and security.

One major limitation is computational complexity. Game theory, when applied to SDN, typically involves solving optimization problems or computing Nash equilibria, tasks that are computationally intensive. When coupled with the real-time decision making required by Snort, the system faces significant challenges in maintaining efficiency, especially in large-scale networks.

In future work, we plan to further explore the scalability of SEC-SDN by integrating game-theoretic models and intrusion detection systems (IDS). These models provide a robust framework for analyzing strategic interactions between attackers and defenders in the network. By incorporating machine learning or data-driven approaches into these game-theoretic models, we aim to approximate optimal outcomes dynamically. This would allow the system to adapt to varying network sizes and the evolving nature of threats, improving its resilience against sophisticated attacks.

Furthermore, enhancing the capabilities of Snort by integrating machine learning or behavior-based analytics could significantly improve its ability to detect and respond to a diverse range of threats. Machine learning would enable Snort to analyze patterns, identify anomalies, and adapt to new attack vectors in real time, making the system more effective against evolving security challenges.

In addition, the use of P4 programming in the framework opens opportunities for real-time threat mitigation. The flexibility of P4 allows immediate actions to be taken directly in the data plane, significantly reducing response times. For instance, if game-theoretic analysis identifies an ongoing attack, P4-programmed devices can implement countermeasures instantaneously, such as rerouting traffic, isolating compromised nodes, or deploying specific packet filtering rules. This real-time adaptability would enhance the framework's effectiveness in managing and mitigating network threats in dynamic environments.

8 Conclusions

The SEC-SDN framework successfully detected and mitigated over 95% of the simulated DDoS attacks with minimal false positives. The integration with OpenStack allowed for seamless management of network security policies.

There was a negligible impact on the performance of legitimate network traffic, demonstrating the efficiency of the framework's threat detection and mitigation algorithms. Testing confirmed that the SEC-SDN framework scales effectively with the increase in cloud infrastructure size and traffic volume, maintaining high levels of security without degrading performance. The framework showed excellent adaptability to the dynamic cloud environment, with the ability to update its configurations in real time based on changing network conditions and the threat landscape.

In conclusion, the SEC-SDN framework has demonstrated significant potential in enhancing the security and stability of network architectures, particularly in cloud environments susceptible to DDoS attacks. By leveraging the flexibility and control offered by software-defined networking, SEC-SDN allows for precise traffic management and rapid response to potential threats. This proactive capability is crucial for detecting and mitigating DDoS attacks, which are often characterized by sudden spikes in traffic that aim to overwhelm systems.

Through the integration of real-time monitoring and dynamic rule application, SEC-SDN has identified unusual traffic patterns indicative of a DDoS attack early in its initiation phase. The ability to immediately redirect or block malicious traffic based on predefined security policies and real-time analysis minimizes the risk of service disruption and maintains the availability of cloud services. Moreover, the SEC-SDN framework facilitates a scalable defense mechanism, essential for protecting large-scale cloud environments where network traffic is voluminous and highly variable.

Furthermore, the adoption of SEC-SDN in cloud infrastructures not only fortifies security measures but also enhances overall network performance. By ensuring that only legitimate traffic is processed by the cloud resources, SEC-SDN prevents unnecessary load on the system, thereby optimizing resource utilization and improving service delivery to end-users. The combination of SEC-SDN with additional security tools like Snort for intrusion detection exemplifies a layered security approach that can effectively shield cloud services from the evolving landscape of cyber threats, including sophisticated DDoS attacks.

Ultimately, the SEC-SDN framework represents a forward-thinking solution to the challenges of network security in the cloud. Its ability to provide comprehensive protection against DDoS attacks while maintaining high levels of network performance is indicative of its potential to become a standard

in the design and operation of secure and resilient cloud networking infrastructures. As cloud technologies continue to advance and form the backbone of modern IT services, the importance of robust security frameworks like SEC-SDN cannot be overstated.

Author Contributions All authors contributed significantly to the development and writing of this paper on the SEC-SDN framework. Specifically, F.R. conceptualized the framework and led the theoretical analysis, conducted the experiments and data analysis, and was responsible for the implementation of the SEC-SDN components. C.M. reviewed related literature and provided critical revisions to the manuscript. All authors discussed the results and implications and commented on the manuscript at all stages.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Neves, R.H., Silva, A.A.A., Gava, V., Azevedo, M.T., Sandoval, J.F.R., et al.: DoS Attack on SDN: a study on control plane strategies in-band and out-of-band. Springer Nature, Berlin (2022)
2. Abedi, Ali, Heard, Andrew, Brecht, Tim: Conducting repeatable experiments and fair comparisons using 802.11n mimo networks. *ACM SIGOPS Oper. Syst. Rev.* **49**(1), 41–50 (2015)
3. Florea, R., Craus, M.: Modeling an Enterprise Environment for Testing Openstack Cloud Platform against Low-Rate DDoS Attacks. In: 26th International Conference on System Theory, Control and Computing (ICSTCC) (2022)
4. Wang, H., Xu, L., Gu, G.: FloodGuard: A DoS Attack Prevention Extension in Software-Defined Networks. In: 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, pp. 239–250. Rio de Janeiro, Brazil (2015). <https://doi.org/10.1109/DSN.2015.27>. Keywords: Switches; Security; Protocols; Software; IP networks; Throughput; Software-Defined Networking; SDN; Security; Denial-of-Service Attack
5. Shin, S., Xu, L., Hong, S., Gu, G.: Enhancing Network Security through Software Defined Networking (SDN). In: 25th International Conference on Computer Communication and Networks (ICCCN), pp. 1–9. Waikoloa (2016). <https://doi.org/10.1109/ICCCN.2016.7568520>. Keywords: Communication networks; Switches; Monitoring; Access control; Centralized control; Software-Defined Networking; Network Security
6. Huang, L., Zhi, X., Gao, Q., Kausar, S., Zheng, S.: Design and implementation of multicast routing system over SDN and sFlow. In: 8th IEEE International Conference on Communication Software and Networks (ICCSN), pp. 524–529 (2016)
7. Arbetu, R.K., Khondoker, R., Bayarou, K., Weber, F.: Security analysis of OpenDaylight, ONOS, Rosemary and Ryu SDN controllers. In: 17th International telecommunications network strategy and planning symposium (Networks), pp. 37–44. IEEE. (PIC S&T) (2016)
8. Tivig, P., Borcoci, E.: Performance evaluation experiments for video and VoIP traffic with RYU controller and mininet framework. *U.P.B. Sci. Bull., Ser. C* **84**(3), 66 (2022)
9. Mamushiane, Lusani, Lysko, Albert, Dlamini, Sabelo: A comparative evaluation of the performance of popular SDN controllers, pp. 54–59. *Wireless Days (WD)*, Paris (2018)
10. Dharma, J.A.: Network attack detection using intrusion detection system utilizing snort based on telegram. *Bit-Tech.* **6**(2), 118–126 (2023)
11. Gala, Y., Vanjari, N., Doshi, D., Radhanpurwala, I.: Hybrid Intrusion Detection System Using Autoencoders and Snort. In: Choudrie, J., Mahalle, P.N., Perumal, T., Joshi, A. (eds.) *ICT with Intelligent Applications. ICTIS 2023. Lecture Notes in Networks and Systems*, vol. 719. Springer, Singapore (2023)
12. Dao, N.N., Park, J., Park, M., Cho, S.: A feasible method to combat against DDoS attack in SDN network. In: *Conference on Information Networking*, pp. 309–311 (2015)
13. Mousavi, S.M., St-Hilaire, M.: Early detection of DDoS attacks against SDN controllers. In: *Conference on Computing, Networking and Communications*, pp. 77–81 (2017)
14. Dharma, N.I.G., Muthohar, M.F., Prayuda, J.D.A., Priagung, K., Choi, D.: Time-based DDoS detection and mitigation for SDN controller. In: *Network Operations and Mag. Symp.*, pp. 550–553 (2015)
15. Shoeb, A., Chithralekha, T.: Resource management of switches and controller during saturation time to avoid DDoS in SDN. In: *IEEE Conference on Engineering and Technology*, pp. 152–157 (2016)
16. Macedo, R., de Castro, R., Santos, A., Ghamri-Doudane, Y., Nogueira, M.: Self-organized SDN controller cluster conformations against DDoS attacks effects. In: *IEEE Global Communications Conference*, pp. 1–6 (2016)
17. Florea, R., Craus, M.: A game-theoretic approach for network security using honeypots. *Future Internet* **14**, 362 (2022). <https://doi.org/10.3390/fi14120362>
18. Jaw, E., Wang, X.: A novel hybrid-based approach of snort automatic rule generator and security event correlation (SARG-SEC). *PeerJ Comput. Sci.* **8**, 66 (2022)
19. Ujjan, R., Pervez, Z., Dahal, K.: Suspicious Traffic Detection in SDN with Collaborative Techniques of Snort and Deep Neural Networks. In: 2018 IEEE 20th International Conference on High Performance Computing and Communications; IEEE 16th International Conference on Smart City; IEEE 4th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), pp. 915–920 (2018)
20. Akhtar, N., Matta, I., Raza, A., Wang, Y.: EL-SEC: ELastic management of security applications on virtualized infrastructure. In: *IEEE INFOCOM 2018—IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pp. 778–783 (2018)
21. Ujjan, R., Pervez, Z., Dahal, K.: Snort Based Collaborative Intrusion Detection System Using Blockchain in SDN. In: 2019 13th International Conference on Software, Knowledge, Information Management and Applications (SKIMA), pp. 1–8 (2019)
22. Paliwal, M., Shrimankar, D., Tembhurne, O.: Controllers in SDN: a review report. *IEEE Access* **6**, 36256–36270 (2018)

23. Zhu, L., Karim, M., Sharif, K., Xu, C., Li, F., Du, X., Guizani, M.: SDN controllers. *ACM Comput. Surv. (CSUR)* **53**, 1–40 (2020)
24. Das, T., Sridharan, V., Gurusamy, M.: A survey on controller placement in SDN. *IEEE Commun. Surv. Tutor.* **22**, 472–503 (2020)
25. Liatifis, A., Sarigiannidis, P., Argyriou, V., Lagkas, T.: Advancing SDN from OpenFlow to P4: a survey. *ACM Comput. Surv.* **55**(9), 1–37 (2022)
26. Goswami, B., Kulkarni, M., Paulose, J.: A Survey on P4 Challenges in Software Defined Networks: P4 Programming. *IEEE Access* (2022)
27. Hauser, F., Häberle, M., Merling, D., Lindner, S., Gurevich, V., Zeiger, F., Frank, R., Menth, M.: A Survey on Data Plane Programming with P4: Fundamentals, Advances, and Applied Research. *ArXiv* (2021)
28. Larsen, J.K., Guanciale, R., Haller, P., Scalas, A.: P4R-Type: A Verified API for P4 Control Plane Programs. *Proceedings of the ACM on Programming Languages* (2023)
29. Ye, C., He, F.: P4b: A Translator from P4 Programs to Boogie. In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (2023)
30. Paolucci, F., Cugini, F., Castoldi, P., Osinski, T.: Enhancing 5G SDN/NFV Edge with P4 Data Plane Programmability. *IEEE Netw.* (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.